

Q-Rex User Manual

1. Installation

Option A — standalone desktop application (no installation of any runtime).

Copy the folder `release/q-rex-<platform>-<arch>/` to the target machine and run `q-rex` (`q-rex.exe` on Windows). No internet connection or Node.js is needed. Build for another platform on a machine with Node ≥ 20 : `npm install && node scripts/build-exe.mjs win32 x64` (or `darwin`, `linux`).

Option B — single file. Open `dist-single/q-rex.html` in Chrome, Edge or Firefox. It is fully self-contained and works offline.

Option C — from source. `npm install`, then `npm run dev` (GUI at <http://localhost:5173>), `npm test` (54 tests), `npm run bench -- 3 20` (headless scenario suite), `npm run e2e` (browser Benchmark-2 check; needs ffmpeg and chromium).

2. Quick start

1. Start the application. The simulation runs immediately: the virtual camera starts at the screen centre, finds the beacon on the wide view, slews to it and locks (state turns green `TRACK`, typically within 1 s).
2. Open the sections in the left panel to change parameters. Disturbance and tracker controls act **live**; scene controls restart the run.
3. Watch the status tiles. A tile turns green when its spec target is met and red when it is missed.
4. At the end of a timed run the performance report (JSON), an HTML report and the per-frame CSV log are saved automatically (browser download folder). Buttons `JSON / CSV LOG / HTML export` at any time.

3. GUI description

Home page (`#/`, opens first): overview, specification-vs-measured table, architecture and state-machine diagrams, innovation tiles, tech stack, benchmark results and the honest limits. *Launch Workstation* opens the workstation (`#/app`); $\leftarrow$ *Home / overview* in the workstation sidebar returns.

Workstation top toolbar: a **LIVE / PAUSED / RUN COMPLETE** badge, chips that summarise the active scenario (motion, size and shape, noise types, jitter, platform, atmosphere, dropouts, seed), the simulation **speed** selector ($\frac{1}{4}\times \dots \text{max}$), **Pause / Run, Restart**, and the **JSON / CSV / HTML** export buttons.

Left panel

- *Show ground truth overlay, Auto-save report at end of run.*
- *Load scenario:* applies a named benchmark scenario (A nominal, B single disturbance at spec maximum, C all maxima, plus dropouts and decoys).
- Collapsible parameter sections (all values clamped to the specification ranges):

Section	Parameters
Scenario & Run	random seed, run duration (0 = endless)
Virtual Camera	camera type (monochrome default / colour), screen size (≥ 2000), camera width/height, FOV (default 4° horizontal), update rate (≥ 30 Hz), max pan/tilt speed $5\text{--}10^\circ/\text{s}$, gimbal acceleration (default $60^\circ/\text{s}^2$; not specified by the statement), extra command latency
Target	motion (straight, circular, figure-8, random, spiral, sinusoidal, user waypoints), speed, size $5\text{--}20$ px, shape, number of targets (1 beacon + decoys), initial location (random / user-defined), waypoints (x,y per line)
Image Noise	salt & pepper (density $\leq 10\%$), Gaussian ($\sigma \leq 20$), Poisson — any combination
Jitter, Platform, Atmosphere	jitter ($\pm \text{px/frame} \leq 20$), platform motion (linear / circular / random / spiral / figure-8, $\leq 20 \text{ px/frame}$), atmosphere (clear, haze, fog, rain, low light) with severity, beacon dropouts, turbulence
Tracker	acquisition mode (wide-view overview / zoom lens / raster scan), zoom speed, control gain, detection threshold, AI verifier (CNN) and its sub-pixel offset option, coast frames

- A yellow warning appears when target speed plus platform speed exceeds the gimbal limit (saturation expected).

Acquisition modes (Tracker section). *Wide-view overview* (default): a $4\times$ -binned view of the whole screen is used only while searching or re-acquiring, the narrow camera tracks. *Zoom lens (single camera)*: no extra sensor — the camera starts at the full-screen FOV ($\approx 16.7^\circ$ so the 4:3 frame covers the whole screen), detects the beacon, then narrows its FOV at the *Zoom speed* while centring; it zooms out again if the beacon is lost. The camera header and the *Lens FOV* telemetry row show the live FOV; errors are always reported in pixels of the configured FOV. In the all-disturbances case it

can need slightly more than 2 s to detect. *Raster scan*: fixed-FOV sweep, cannot meet 2 s ($\approx$ 5–13 s).

Status bar — current state (SEARCH, SLEW, TRACK, COAST, REACQUIRE) and tiles: acquisition time ($\leq$ 2 s), mean tracking error with RMSE/max ($\leq$ 10 px), centroid error RMSE/P95, target loss and lock retention ($<$ 5 %), maximum re-acquisition time ($\leq$ 1 s), tracker FPS and ms/frame ($\geq$ 20 FPS). Green = met, red = missed.

Dashboard (2 × 2)

- *Virtual camera* (top left): the 640×480 image with overlays — red cross = boresight, green square = detection, yellow circle = filter estimate, cyan dashed box = search window, magenta circle = ground truth (optional). Shown at most at native size so it stays sharp.
- *Screen map* (top right): the whole 2000×2000 screen as a true square with the camera footprint (blue box), beacon trail and dot (magenta) and the estimate (green).
- *Live error* (bottom left): boresight error of the last 300 frames against the 10 px spec band, with a colour strip of tracker state. The line starts once the beacon is first centred; the y-axis rescales automatically.
- *Tracker telemetry* (bottom right): state, sensing mode, camera pointing, estimated beacon speed, CV/CA model mix, filter position σ , learned measurement noise, search window, processing time and gimbal saturation.

Endless runs (`duration = 0`): with *Auto-save* on, JSON, HTML and CSV are downloaded every 60 s of simulated time, and the browser asks for confirmation before the window is closed with unsaved frames.

4. Benchmark 1 — scenario suite

Set seconds per run and number of seeds, press *Run all scenarios*. Each scenario runs headlessly; the table shows acquisition, error, centroid RMSE, loss, lock, re-acquisition, FPS and pass counts (green = all runs met the spec check). *Export suite JSON* saves the full metrics of every run. The same suite runs without the GUI: `npm run bench -- <seeds> <seconds> [filter]`.

5. Benchmark 2 — video input

1. In the *Benchmark 2* panel (step 1) choose the `.mp4` (full screen of any size, or a 640×480 camera image). Step 2 (optional): choose a ground-truth CSV — rows

`frame,x,y` OR `time_s,x,y`, comma / semicolon / tab separated, header optional, 0- or 1-based frames auto-detected — and select the *coordinate convention* — whether its coordinates are *pixel index* (centre of the first pixel = 0) or *pixel edge* (= 0.5). Step 3: set the video frame rate (30) and optionally tick *AI verifier (CNN)*.

2. Press *Run on video* (a progress bar shows the decoding; *Stop* cancels). The PTZ model is bypassed: every frame is decoded exactly (seek per frame), converted to grey, and passed to the same detector/tracker logic. The preview shows the detection.
3. Six result cards show acquisition, lock retention, centroid RMSE, tracker FPS, maximum re-acquisition and frame count; *All metrics* expands the full table. It gives frames, processing FPS, acquisition time, lock retention, re-acquisition events/time, and — with ground truth — centroiding error mean/RMSE/P95/max. *Centroid log CSV* contains `frame,t,found,x,y,state,procMs` per frame; coordinates use the OpenCV pixel-index convention (centre of the first pixel = 0.0).

Frames ≥ 1000 px are treated as the whole screen: search runs on a 4× binned copy, tracking on a window at full resolution.

6. Performance report fields

`simulationDurationSec`, `averageFPS`, `endToEndFPS`, `processingTimeMeanMs/P95/Max`, `acquisitionTimeSec`, `timeToCentreSec`, `trackingErrorMeanPx/RmsePx/P95/P99/MaxPx`, `trackingErrorInclJitterMeanPx`, `centroidingError...`, `lockRetentionRatePct`, `targetLossPct`, `reacquisition...`, `gimbalSaturationPct`, per-requirement pass/fail checks, and the complete configuration used (so a run can be reproduced from the seed).

7. Troubleshooting

- *Acquisition > 2 s*: the *Raster scan* mode cannot meet 2 s by physics (12 s sweep); use *Wide-view overview* or *Zoom lens*. Zoom lens works under salt & pepper noise too (a switching median keeps the zoomed-out beacon); only the all-disturbances case is slower.
- *Red tracking-error tile with platform + jitter at maximum*: expected limitation (see technical report §9).
- *Video fails to load*: the browser must be able to decode the codec (H.264 is supported by Chrome/Edge/Electron).
- *Slow GUI*: set Sim speed to 1×, or use the headless `npm run bench`.
- *Check a desktop build*: `q-rex --selftest` (add `--selftest-shot=file.png` for a screenshot) renders the home page and the workstation, checks that the tracker locks, prints a

JSON line and exits 0 on success. The GitHub workflow runs it on Windows, macOS and Linux.